# Working With Voice Streaming

❖ Voice streaming in VI is designed as a flow-driven capability within the VI DIY no-code platform, rather than an API feature.

**Flow-Based Configuration**

All streaming behaviour / call flow is configured directly within the VI CPaaS DIY Flow designer using the Streaming Object.

**Users can:**

- Configure WebSocket (WSS) endpoints directly from the UI.
- Choose between Unidirectional and Bidirectional streaming.
- Define streaming duration and execution mode (Foreground or Background).
- Configure dynamic WebSocket URLs using variables and API responses.
- Pass custom parameters to the streaming endpoint.
- Integrate streaming with Incoming call flows, IVR menus, agent routing, call patching, and other call-flow components without writing code.

**APIs Are Used for Call Initiation**

VI APIs are primarily used to initiate or trigger calls, including:

- Authentication
- Campaign management
- Outbound Calls
- Click-to-Call

Once the call enters the configured flow, streaming behaviour / call flow is handled by the flow configuration and Streaming Object settings.

---

## ❖ Types of Streams

VI supports two types of WebSocket-based call streams: **Unidirectional** and **Bidirectional**.

**1. Unidirectional Streams**

- **Direction**: Audio flows in one direction (VI → WebSocket Endpoint).
- **Purpose**: The call audio is streamed live to your endpoint for real-time processing.
- **Examples of Use Cases**:
- Live speech-to-text transcription
- Real-time monitoring of agent calls
- Supervisor coaching during calls

Only the **caller's audio** is streamed; no audio is sent back to the caller.

---

**2. Bidirectional Streams**

- **Direction**: Audio flows both ways (VI ↔ WebSocket Endpoint).
- **Purpose**:
- VI sends the caller's audio to your endpoint.
- Your endpoint sends audio responses back, which VI plays to the caller.
- **Examples of Use Cases**:
- Building interactive voice bots (AI-powered assistants)
- Dynamic, conversational IVR systems

Enables **two-way real-time conversation** over WebSocket.

---

❖ **Important Note on Recording**

Voice streaming recordings are available for a period of 30 days. Recording URLs can be retrieved through the API or reports. To ensure continued access, please download the files within this 30-day retention window.

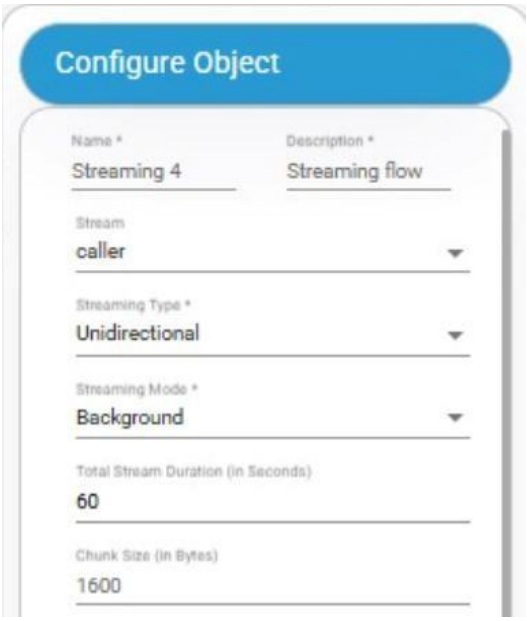
**Chunk Size Requirements**

When enabling streaming, the chunk size must follow strict guidelines:

- **Minimum size**: **1.6 KB (around 100 ms of audio)**
- Smaller chunks may cause **jitter or broken audio** due to network instability.
- **Maximum size**: **50 KB**
- Larger chunks may lead to **timeouts** or delayed playback.
- **Must be a multiple of 160 bytes**
- Each packet represents **10 ms of audio = 160 bytes**.

❖ **Configuring a Unidirectional Stream – Streaming Object**

**1. Enable unidirectional streaming on a call flow using Streaming Object**

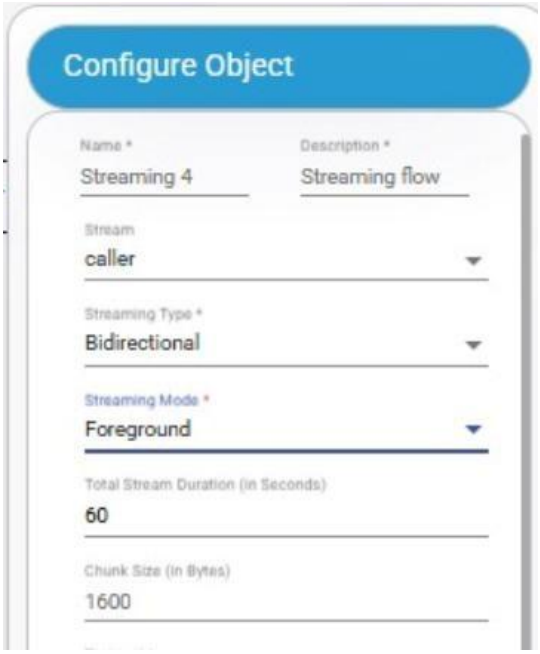



Unidirectional streaming can be enabled by selecting streaming type “Unidirectional”. Streaming mode can be selected from the Foreground/Background. If Foreground selected, the call flow will not move to the next object and the stream ends if the call disconnects, the WebSocket closes, the client explicitly stops it, or the “Total stream duration” elapses. If Background selected, stream would be created, and the call flow would be moved to the next flow object.

**2. Enable unidirectional streaming on a call using API**

Unidirectional streaming can start by triggering our standard API(s) for all types of calls like incoming, outgoing, patching and Click-to-Calls. Refer to API documentation for more details.

❖ **Configuring a Bidirectional Stream on a call flow– Streaming Object**



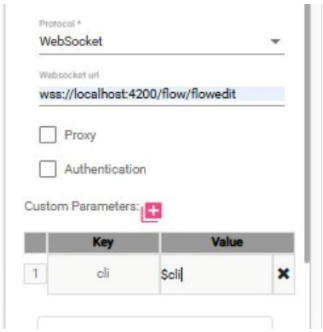

Bidirectional streaming can enable by selecting streaming type “Bidirectional”. Streaming mode will be always Foreground hence the stream can end if the call is disconnected or the WebSocket is closed, or the stream is explicitly stopped by the client or the “Total Stream Duration” completed.

❖ **VI Call Streaming Protocol (WebSocket Based)**

Communication between VI and a customer endpoint happens over a WebSocket connection.

WebSocket URL can be configured in streaming object, and we will only support WSS. A user can enter static URL or dynamic URL. For configuring a dynamic URL, users can put a variable in WebSocket URL field, which can be fetched using an API in the flow from a customer end point. There is a key value pair configuration (maximum of 3 parameters)

also available which can be used as custom parameters that will be passing through streaming start event body.




All messages are exchanged as JSON strings.

## WebSocket Messages: From VI → Customer Endpoint

The following events can be received:

- connected
- start
- media
- dtmf
- stop
- mark (only in bidirectional streaming)
- clear

**1. Connected Message**

Sent immediately after a WebSocket connection is established.

```
{
  "event": "connected"
}
```

**2. Start Message**

Sent right after the connected message (only once).
Includes stream details and any custom parameters passed in the Stream Applet URL.

Example:
If the configured URL is:

wss://yourstream.service.com

Then queuename and product will be passed as key–value pairs (queuename=premium & product=radio)


```
{
    "event": "start",
    "sequence_number": 1,
    "room_id": "<room id>",
    "start": {
      " room_id ": "<room id>",
      "call_id": "<call id>",
      "cli": "<caller number>",
      "dni": "<recipient number>",
      "custom_parameters": {
        "queuename": "premium",
        "product": "radio"
      },
      "media_format": {
        "encoding": "raw",
        "sample_rate": "8000",
        "bit_rate": "128000"
      }
    }
  }
```

**3. Media Message**

Encapsulates audio packets in base64 format.

- media.chunk → Chunk index in sequence
- media.timestamp → Offset in ms from start of stream
- Audio format → 16-bit linear PCM, 8kHz, PCM16, 128 Kbps, base64 encoded

For bidirectional mode, the client must send audio back in the same format for playback to the human caller.

```
{
  "event": "media",
  "sequence_number": 3,
  "room_id": "<room id>",
  "media": {
              "chunk": 2,
              "timestamp": "10",
              "payload": "<base64-encoded PCM>"
            }
}
```

**4. DTMF Message**

Sent when a caller presses a digit on their keypad (supported only for bidirectional streams in Voice Bot flows).

```
{
  "event": "dtmf",
  "sequence_number": 1,
  "room_id": "<room id>",
 "dtmf": {
```

```
            "duration": "<duration in ms>",
            "digit": "<pressed digit>"
          }
}
```

**5. Stop Message**

Indicates that the stream has stopped or the call has ended.

```
{
  "event": "stop",
  "sequence_number": 10,
  "room_id": "<room id>",
"stop": {
            "call_id": "<call id>",
            "reason": "stopped or call ended"
          }
}
```

**Note**: If the IVR server receives malformed data, data of unsupported size, or an unsupported data format from a WebSocket endpoint, the system will terminate the audio stream by sending a Stop message with reason and promptly disconnect the corresponding IVR call.

**6. Mark Message (Only in Bidirectional Streaming)**

Used as a checkpoint for media playback completion.

```
{
  "event": "mark",
```

```
 "sequence_number": 15,
"room_id": "<room id>",
        "mark": {
          "name": "<label>"
        }
}
```

## WebSocket Messages: From Customer Endpoint → VI

These apply only in bidirectional streaming mode.

**1. Media Message (to VI)**

Send audio data back to VI in base64 PCM format:

```
{
  "event": "media",
  "sequence_number": 3,
  "room_id": "<room id>",
  "media": {
          "chunk": 2,
          "timestamp": "10",
          "payload": "<base64-encoded PCM>"
        }
}
```

**2. Mark Message (to VI)**

Client can send a mark to request acknowledgment when specific audio has finished playing. VI will respond with a matching mark event once completed.

```
{
  "event": "mark",
  "sequence_number": 15,
  "room_id": "<room id>",
  "mark": {
            "name": "<label>"
          }
}
```

**3. Clear Message**

Used to clear previously sent audio that has not yet been played out.

```
{
  "event": "clear",
  "room_id": "<room id>"
}
```

**Note**: Sending media in smaller chunks helps ensure the Clear event works as intended. Example: If 2 audio messages of 5s each are queued, and you send clear at the 3rd second → only the second chunk will be cleared if the first is already playing.

**4. Exit Message**

The Exit Message is used within WebSocket communications to request the termination of an active streaming session. When this message is sent, the server initiates the process to safely close the WebSocket stream associated with the specified room ID. This ensures that the session is terminated gracefully and allows the IVR flow to continue with any subsequent call logic.

In addition to terminating the session, the Exit Message can now carry custom parameters that can be consumed by the IVR/application flow after streaming ends. This enables use cases such as agent transfer, call disposition handling, routing decisions, or passing transaction-specific information.

```
{
  "event":"exit",
  "room_id":"<room_id>",
  "exit”: {
    "parameters”: {
      "name":"qwerty",
      "id":2345678
    }
  }
}
```

---

**Media Format**

- Format: raw (16-bit linear PCM, 8kHz, PCM16, 128 Kbps)
- Encoding: Base64
- Both inbound (caller audio) and outbound (bot/caller response) must adhere to this format in bidirectional streaming.

**5. Call patching with an Agent**

Vi supports two mechanisms for determining the next action after a media streaming session ends. Customers can choose either of the following approaches based on their integration requirements.

**Option 1: Exit Parameters**

The customer can include custom parameters in the WebSocket exit event. Once the streaming session is terminated, these parameters are made available to the IVR/application flow and can be used for routing decisions, agent transfer logic, transaction tracking, or other business-specific actions.

Example


```
{
  "event": "exit",
  "room_id": "HB_41117619_0_1411607",
  "exit": {
    "parameters": {
      "action": "transfer",
      "agent_number": "9876543210",
      "case_id": 2345678
    }
  }
}
```


**Notes:**

Up to 10 custom parameters can be passed.

The IVR/application flow can consume these parameters to determine the next action after streaming ends.

---

**Option 2: API**

The customer can configure an API that will be invoked by Vi once the streaming session ends, either through an exit event or due to stream disconnection.

Vi sends the call details to the customer API, and the API response determines whether the call should be transferred to an agent or disconnected.

Sample URL

https://XXXXXXXX/XXXXXXX/api/Data/json

The customer must provide the API endpoint URL.

**Request Payload**


{

"Callid": "XXXXX",

"dni": "XXXXXX",

"cli": "XXXXXX"

}


**Response**


{

"callstatus": "1",

"rm_number": "9876543210"

}


Response Parameters

| Parameter | Value | Description |
|---|---|---|
| callstatus | 1 | Transfer the call to the specified agent number |
| callstatus | 0 | Disconnect the call |
| rm_number | 10-digit number | Agent number to which the call should be transferred (mandatory when callstatus = 1) |

**Flow**

Streaming session ends.

Vi invokes the customer API.

Customer API returns the next action.

Vi either transfers the call to the specified agent number or disconnects the call based on the response.